

Git & GitHub Beginner Guide

A practical reference for uploading an existing project, working from two PCs, and solving common problems

Who this guide is for: Someone starting with little or no knowledge of Git or GitHub and who wants version history plus a reliable way to synchronize code between multiple computers.

The Big Idea

Git tracks the history of your files on your computer. GitHub stores a copy of that repository online. Together, they let you save checkpoints, recover history, and synchronize work between PCs.

Term	Meaning
Git	Tracks changes and history locally.
GitHub	Hosts repositories online.
Commit	A saved checkpoint in project history.
Push	Uploads commits to GitHub.
Pull	Downloads the latest changes from GitHub.
Clone	Downloads an existing repository for the first time.

Your Normal Two-PC Workflow

The safest habit is: **pull before starting work, then commit and push when you finish.**

```
PC 1: git pull → work → git add . → git commit -m "message" → git push
```

```
PC 2: git pull → work → git add . → git commit -m "message" → git push
```

Avoid editing the same file differently on both PCs before synchronizing. Otherwise Git may require you to resolve a merge conflict.

Part 1 — First-Time Setup on Your Main PC

Use this when you already have a folder full of code and want to begin tracking it with Git.

1. Install Git

```
git --version
```

If a version number appears, Git is installed.

2. Configure your name and email

```
git config --global user.name "Your Name"
git config --global user.email "your-email@example.com"
```

If you want the identity only for the current project, omit **--global**. Check your configuration with:

```
git config --global user.name
git config --global user.email
```

3. Open your project folder

```
cd D:\samples
dir
```

Confirm that you are inside the folder containing the code you want to upload.

4. Initialize Git

```
git init
git branch -M main
```

This creates a hidden .git directory containing local history and configuration.

5. Create a .gitignore before adding everything

```
.env
.venv/
venv/
__pycache__/
*.pyc
node_modules/
```

Never intentionally upload passwords, API keys, private tokens, or secret configuration files.

6. Create your first commit

```
git status
git add .
git status
git commit -m "Initial commit"
```

Part 2 — Connect the Project to GitHub

Create a new empty repository on the GitHub account that should own the project. For a simple first upload of an existing folder, do not initialize the GitHub repository with a README, .gitignore, or license.

Connect the local folder to GitHub

```
git remote add origin https://github.com/YOUR-USERNAME/YOUR-REPOSITORY.git  
git remote -v
```

Push the first version

```
git push -u origin main
```

A successful result includes a message similar to:

```
* [new branch] main -> main  
branch 'main' set up to track 'origin/main'
```

This confirms that the upload succeeded and future git push / git pull commands know which GitHub branch to use.

Part 3 — Set Up the Second PC

Do not normally run `git init` again for this same GitHub project. Instead, clone the repository.

```
cd D:\projects
git clone https://github.com/YOUR-USERNAME/YOUR-REPOSITORY.git
cd YOUR-REPOSITORY
```

The project files and Git history are now available on the second PC.

Before beginning work

```
git status
git pull
```

Ideally, your working directory should be clean before switching computers or starting a new session.

Part 4 — Your Daily Workflow

Before starting work:

```
git status
git pull
```

After making changes:

```
git status
git add .
git commit -m "Describe what you changed"
git push
```

Examples of useful commit messages:

```
git commit -m "Added image preprocessing"
git commit -m "Fixed login validation"
git commit -m "Updated project documentation"
```

If you are unsure what Git is doing, run **git status**. It is the most useful diagnostic command for beginners.

Part 5 — Common Problems and Self-Solutions

Problem 1: fatal: unable to auto-detect email address

Cause: Git does not know the identity to attach to commits.

```
git config --global user.name "Your Name"
git config --global user.email "your-email@example.com"
git add .
git commit -m "Initial commit"
```

Problem 2: src refspec main does not match any

Common cause: You tried to push before making your first commit.

```
git add .
git commit -m "Initial commit"
git branch -M main
git push -u origin main
```

Problem 3: Permission denied / error 403

This usually means the GitHub account currently authenticated on the PC does not have permission to push to the repository, or the remote URL points to the wrong account/repository.

```
git remote -v
```

If the URL is wrong:

```
git remote set-url origin https://github.com/CORRECT-USERNAME/CORRECT-REPOSITORY.git
```

If the URL is correct but the wrong account is cached, remove the old GitHub credentials from the operating system's credential manager and push again. Authenticate with the account that owns the repository or has write permission.

Problem 4: error: remote origin already exists

You already added a remote named origin. Check it instead of adding it again:

```
git remote -v
```

Change its URL if needed:

```
git remote set-url origin NEW-REPOSITORY-URL
```

Part 6 — Merge Conflicts

A merge conflict can happen when two versions change the same part of a file.

```
git status
```

A conflicted file may contain markers like:

```
<<<<<< HEAD
Your version
=====
Other version
>>>>>> origin/main
```

Edit the file so it contains the correct final content and remove the markers. Then:

```
git add FILE-NAME
git commit -m "Resolved merge conflict"
git push
```

The easiest prevention is to finish work on one PC, commit and push it, then pull before continuing on the other PC.

Part 7 — Starting Fresh When the Local Git Setup Is Wrong

If you intentionally want to discard only the local Git configuration and history while keeping your ordinary project files, make sure you are in the correct project folder and remove the hidden `.git` directory.

```
PowerShell:
Remove-Item -Recurse -Force .git
```

Then verify:

```
git status
```

Git should report that the folder is not a Git repository. You can then repeat the initial setup.

Warning: Do not casually delete `.git` from a repository with history you want to preserve. A safer alternative is often to clone the correct repository into a new folder.

Part 8 — Command Cheat Sheet

Command	Purpose
git status	See changed, staged, and untracked files.
git pull	Get the latest changes from GitHub.
git add .	Stage current changes.
git commit -m "message"	Create a saved checkpoint.
git push	Upload local commits to GitHub.
git clone URL	Download an existing repository for the first time.
git remote -v	Check the configured repository addresses.
git log --oneline	View a compact commit history.

Recommended Safety Rules

1. Run **git status** when unsure what changed.
2. Run **git pull** before starting work on another PC.
3. Commit changes in logical groups with descriptive messages.
4. Push when you finish so the latest work is available elsewhere.
5. Never upload secrets, passwords, API keys, or private .env files.
6. Run **git remote -v** if you suspect the project is connected to the wrong repository.
7. Do not run **git init** inside an already cloned project unless you know why.
8. Do not delete .git unless you intentionally want to remove local Git history/configuration.

Final Reference Workflow

```
FIRST PC — FIRST TIME
1. Configure Git name/email
2. cd into project folder
3. git init
4. Create .gitignore
5. git add .
6. git commit -m "Initial commit"
7. Create empty GitHub repository
8. git remote add origin REPOSITORY-URL
9. git branch -M main
10. git push -u origin main
```

```
SECOND PC — FIRST TIME
1. Install/configure Git
2. cd to desired location
```


3. `git clone REPOSITORY-URL`
4. `cd project-folder`

EVERY DAY ON EITHER PC

1. `git status`
2. `git pull`
3. Work on code
4. `git status`
5. `git add .`
6. `git commit -m "Describe changes"`
7. `git push`

When something fails, avoid repeatedly trying random commands. Start with **git status**, read the exact error, and use **git remote -v** for repository connection or account problems.